CLOUD COMPUTING · AULA 13 · 12 DE NOVEMBRO DE 2026

A loja aguenta o pico

Escalabilidade, resiliência e observabilidade: quanto a loja serverless aguenta, como uma fila protege o que não escala, e como saber que algo deu errado antes do cliente reclamar.

Faculdade Senac Pernambuco · ADS 2026.2 · Prof. Sebastião Rogério

ONDE ESTAMOS NO SEMESTRE · 1 DE 3

O caminho até aqui

U1 · ✔ CONCLUÍDA

Fundamentos e economia

20/08
O que é nuvem (NIST)

27/08
Camadas de serviço

03/09
Economia e migração

U2 · ✔ CONCLUÍDA

Setup e computação

10/09
Regiões e o primeiro recurso no S3

17/09
EC2 na prática

U3 · ✔ CONCLUÍDA

Serviços e APIs

24/09
AV1 e serviços via API

01/10
Serverless: Lambda

08/10
API Gateway

15/10
Gerenciar a API

U4 · ✔ CONCLUÍDA

Dados distribuídos

22/10
Armazenamento distribuído

29/10
Banco NoSQL

U5 · ◀ ESTAMOS AQUI

Rede e arquitetura

05/11
Rede em nuvem

12/11
Hoje: escala e observabilidade

19/11
ARQUITETURA FINAL

26/11
AV2

ONDE ESTAMOS NO SEMESTRE · 2 DE 3

A arquitetura da loja, peça por peça

Navegador → S3	a vitrine · 10/09	✔ pronto
API Gateway	rotas, estágio v1, limite e chave · 08/10 e 15/10	✔ pronto
Lambda	as regras da loja · 01/10	✔ pronto
+ classes e ciclo de vida	arquivos baratos com o tempo · 22/10	✔ pronto
Banco NoSQL	os dados saem do código · 29/10	✔ pronto
Rede (VPC)	sub-rede privada e endpoint · 05/11	✔ pronto
Fila, escala e alarme	aguentar o pico e saber quando quebra · 12/11	◂ HOJE

- A loja tem vitrine, API, regras, banco e rede. Tudo funciona **num dia normal**.
- Hoje a pergunta muda: e no dia em que chegam **trinta vezes mais pedidos**? E quando uma peça falhar, quem fica sabendo?
- É a última peça antes da arquitetura final de 19/11.

ONDE ESTAMOS NO SEMESTRE · 3 DE 3

Uma semana para a arquitetura final

DATA	AULA	TEMA	O QUE ACRESCENTA À ARQUITETURA
05/11	12	Arquitetura de rede	sub-rede pública e privada, endpoint do banco
12/11	**13**	**Escalabilidade, resiliência e observabilidade**	**hoje: fila, carga e alarme**
19/11	14	**ARQUITETURA FINAL**	apresentação com demonstração ao vivo e arguição: 50% do 2º processo
26/11	15	**AV2** e devolutiva	prova: 50% do 2º processo
03/12 e 10/12	16 e 17	Multicloud e encerramento	conteúdo complementar e devolutiva, sem nota

O QUE VAMOS VER HOJE

A loja na Black Friday

MUNDO 1

Escalar

conceitos com contas

- O pico
- Como a Lambda escala
- O elo mais fraco

MUNDO 2

Resistir e enxergar

conceitos com exemplos

- A fila
- Quando falha
- Logs, métricas e alarmes

MUNDO 3

Prática

vocês seguem a aula

- A fila da loja
- O pico na fila
- O alarme

FECHAMENTO

Arquitetura final

em equipe

- O componente da equipe
- O roteiro de 19/11



O MAPA DE HOJE

Nove fases, três mundos

Mundos 1 e 2: escala, fila, falhas e observabilidade, com as contas da loja. Mundo 3: os pedidos da loja passando por uma fila na AWS, um teste de carga e um alarme que chega por e-mail. Tudo dentro da faixa gratuita.



FASE 1-1 · CONTEÚDO NOVO

O pico

Uma retomada rápida da Aula 12 e o dia em que a loja vende trinta vezes mais.

RELEMBRE AULA 12

A loja com rede

A função **loja-api** mora nas **sub-redes privadas** da `loja-vpc`, sem saída para a internet. Ela alcança o DynamoDB por um **gateway endpoint**, grátis, e o caminho até o banco nunca passa pela internet. Nenhum NAT Gateway: a conta da rede fechou em zero.

exemplo

```
internet
  └ API Gateway  loja-api · estágio v1 · x-api-key
      └ Lambda loja-api   sub-redes privadas 1a e 1b
          └ endpoint      vpce-... (Gateway)
              └ DynamoDB  loja-produtos

NAT Gateway: nenhum
```

CUIDADO

Uma função na sub-rede privada **só alcança o que tem rota**. Hoje entra um serviço novo, a fila, e essa regra vai decidir onde cada função mora.

FASE 1-1 · DIAGNÓSTICO DA AULA 12

Três perguntas rápidas sobre a rede da loja

- a) O que faz uma sub-rede ser pública?
- b) Por que a função parou de responder antes de criar o endpoint?
- c) Por que o NAT Gateway ficou de fora?

a) Uma rota `0.0.0.0/0` para o Internet Gateway na tabela de rotas dela.

b) A sub-rede privada não tinha caminho para o DynamoDB: a chamada esperava até estourar o tempo limite.

c) Ele cobra por hora e por GB, e a loja não precisa sair para a internet: o endpoint resolve o banco de graça.

FASE 1-1 · O PROBLEMA

Num dia normal, a loja recebe 2 pedidos por segundo. Na Black Friday, recebe 60.

Arquitetura não se prova no dia comum. A pergunta desta aula é o que acontece com cada peça da loja quando o movimento multiplica por trinta, e o que acontece quando uma delas falha no meio do pico.

CONCEITO AULA 13

Escalar na vertical e na horizontal

Escalar na vertical é trocar por uma máquina maior: mais CPU, mais memória. Tem teto e geralmente exige parar. **Escalar na horizontal** é pôr mais cópias trabalhando lado a lado. Não tem teto prático, mas só funciona se as cópias **não guardam estado** entre um pedido e outro. É a **elasticidade** da definição do NIST, da Aula 1.

exemplo

```
vertical     1 servidor t3.small → t3.2xlarge
             mais forte, mas um só

horizontal   1 cópia → 30 cópias iguais
             cada pedido vai para qualquer uma

Lambda       horizontal, automático:
             uma cópia nova por pedido simultâneo
```

PENSE ASSIM

Vertical é contratar um caixa mais rápido. Horizontal é abrir mais caixas. No dia de pico, a fila anda porque abriram caixas, não porque o caixa ficou mais rápido.



FASE 1-2 · CONTEÚDO NOVO

Como a Lambda escala

Concorrência: quantas cópias da função trabalham ao mesmo tempo, e qual é o teto.

CONCEITO AULA 13

Concorrência

A Lambda escala sozinha, mas não sem limite. O número que importa é a **concorrência**: quantas execuções acontecem **ao mesmo tempo**. Ela sai de uma conta simples, a **lei de Little**: pedidos por segundo vezes a duração de cada um. Cada execução simultânea é uma cópia da função na memória.

a conta

```
concorrência = pedidos por segundo x duração

dia normal     2/s  x 0,5 s =   1 cópia
Black Friday  60/s  x 0,5 s =  30 cópias
se a função demorar 3 s:
              60/s  x 3 s   = 180 cópias
```

CUIDADO

A duração pesa tanto quanto o movimento. Uma função que espera um serviço lento **ocupa a cópia** o tempo todo: deixar a função mais rápida também é escalar.

CONCEITO AULA 13

O teto da conta

Toda conta tem uma **cota de concorrência** por região, dividida entre **todas** as funções. O padrão é 1.000; contas novas costumam começar com bem menos, e o valor aparece em **Service Quotas**. Passou do teto, a Lambda recusa: é o *throttling*, a estrangulação, e o API Gateway devolve **429** ou **503** para quem chamou.

no CloudShell

```
aws lambda get-account-settings \
  --query "AccountLimit.ConcurrentExecutions"
1000        ← ou bem menos, em conta nova

aws lambda get-account-settings \
  --query "AccountLimit.UnreservedConcurrentExecutions"
```

PENSE ASSIM

O teto é da conta, não da função: uma função descontrolada pode consumir as cópias de todas as outras. Por isso existe a **concorrência reservada**, que separa uma fatia para uma função.

FASE 1-2 · FAÇA A CONTA

Quantas cópias ao mesmo tempo?

- a) 20 pedidos por segundo, cada um levando 0,2 s.
- b) 20 pedidos por segundo, cada um esperando 4 s por um serviço externo.
- c) A conta tem teto 10. O que acontece no caso b?
- d) Como reduzir o caso b sem mexer no movimento?

a) 4 cópias: 20 × 0,2.

b) 80 cópias: 20 × 4. O movimento é o mesmo; a espera multiplicou por vinte.

c) A partir da 11ª execução simultânea, a Lambda recusa: o cliente recebe erro, mesmo com a função funcionando.

d) Tirar a espera do caminho: o pedido entra, a resposta sai na hora, e o trabalho lento fica para depois. É a fila da fase 2-1.



FASE 1-3 · CONTEÚDO NOVO

O elo mais fraco

A Lambda escala. O resto da cadeia, nem sempre.

CONCEITO AULA 13

Acoplamento síncrono

Num pedido **síncrono**, quem chamou espera a cadeia inteira responder. A velocidade da loja vira a velocidade do **elo mais lento**, e a falha de qualquer peça vira falha para o cliente. Isso é **acoplamento**: uma peça depende de a outra estar de pé, e rápida, naquele exato momento.

o pedido de compra hoje

```
cliente → API → Lambda → estoque     0,1 s
                       → pagamento   2,0 s
                       → nota fiscal 1,5 s
                                     ------
            o cliente espera         3,6 s

a nota fiscal aceita 5 por segundo
na Black Friday chegam 60 → 55 erros por segundo
```

CUIDADO

A Lambda escalar não ajuda: ela abre 60 cópias, e as 60 batem juntas num serviço que só aguenta 5. Escalar a ponta mais forte **derruba a mais fraca**.

FASE 1-3 · NA IMAGEM

Quem escala e quem não escala na loja

ESCALA SOZINHO

API Gateway, Lambda, S3

Gerenciados e horizontais. O limite existe, mas é alto, é conhecido e aparece numa cota.

ESCALA SE CONFIGURADO

DynamoDB

No modo **on-demand**, da Aula 11, acompanha o movimento. No modo provisionado, recusa o que passar da capacidade contratada.

NÃO ESCALA

O mundo de fora

O pagamento, a nota fiscal, o e-mail, o sistema antigo da cliente. Têm limite fixo e você não controla.

A pergunta de arquitetura não é "a minha função escala?". É "qual peça quebra primeiro, e o que acontece com o pedido quando ela quebra?".

Na Black Friday, qual peça quebra primeiro?

- a) A vitrine no S3, com 60 acessos por segundo.
- b) A função que calcula o carrinho, levando 0,1 s.
- c) A chamada ao emissor de nota fiscal, limitado a 5 por segundo.
- d) A tabela de produtos no DynamoDB, em on-demand.

c) a nota fiscal. É a única peça com limite fixo e fora do seu controle.

a) O S3 serve arquivos estáticos em escala muito maior que essa.

b) 60 × 0,1 = 6 cópias: confortável.

d) On-demand acompanha o movimento. Seria problema no modo provisionado com capacidade baixa.



FASE 2-1 · CONTEÚDO NOVO

A fila

Desacoplar: o pedido entra na hora, e o trabalho lento acontece no ritmo de quem consegue.

CONCEITO AULA 13

Fila: produtor e consumidor

Uma fila guarda mensagens entre quem produz e quem consome. O **produtor** põe a mensagem e segue a vida; o **consumidor** retira e processa no ritmo dele. Na AWS, a fila gerenciada é o **SQS**, *Simple Queue Service*. O pico não desaparece: ele vira **fila acumulada**, que esvazia aos poucos, em vez de virar erro.

o pedido de compra com fila

```
cliente → API → Lambda porta → fila     0,05 s
          ← 202 "pedido recebido"

      fila → Lambda consumidora → pagamento
                                → nota fiscal
      no máximo 5 por vez: o ritmo da nota fiscal

pico: 60 entram por segundo, 5 saem por segundo
a fila acumula e depois esvazia
```

PENSE ASSIM

É a senha da padaria no domingo: todo mundo pega senha na hora, ninguém é mandado embora, e o balcão atende no ritmo que consegue.

CONCEITO AULA 13

202: recebido não é concluído

Com a fila, a API responde **202 Accepted**, aceito: o pedido foi registrado, mas ainda não foi processado. Isso muda a conversa com o cliente: a tela diz "pedido recebido" e o resultado chega depois, por e-mail ou numa consulta de situação. É uma decisão de produto, não só de infraestrutura.

a resposta da porta

```
POST /pedidos
{"produto": 2, "quantidade": 1}

202 Accepted
{"pedido": "a91f03c2", "status": "na fila"}

// 201 Created seria mentir: ainda não foi criado
```

CUIDADO

Nem tudo pode ir para a fila. Consultar o preço ou o carrinho precisa de resposta **na hora**. Fila serve para o que pode terminar depois: pagamento, nota, e-mail, relatório.

CONCEITO AULA 13

Entrega pelo menos uma vez

A fila padrão do SQS garante que a mensagem é entregue **pelo menos uma vez**, e a ordem é aproximada. Raramente, a mesma mensagem chega duas vezes. Por isso o consumidor precisa ser **idempotente**: processar o mesmo pedido duas vezes tem que dar o mesmo resultado que processar uma.

exemplo

```
fila padrão   entrega pelo menos uma vez
              ordem aproximada
              vazão praticamente sem limite

fila FIFO     entrega exatamente uma vez
              ordem garantida
              vazão limitada

idempotente: "se o pedido a91f já foi pago, não paga de novo"
```

CUIDADO

O `id` do pedido é a defesa: antes de cobrar, o consumidor confere se aquele `id` já foi processado. É a mesma ideia do `attribute_not_exists(id)` da Aula 11.



FASE 2-2 · CONTEÚDO NOVO

Quando falha

Tentar de novo, desistir com elegância, e não perder o pedido.

CONCEITO AULA 13

Invisível, não apagada

Quando o consumidor pega uma mensagem, ela não sai da fila: fica **invisível** por um tempo, o *visibility timeout*. Se o consumidor terminar, apaga a mensagem. Se falhar ou sumir, o tempo acaba e ela **volta a aparecer** para uma nova tentativa. É assim que a fila não perde pedido quando uma função quebra no meio.

a vida de uma mensagem

```
entra na fila                    visível
consumidor pega                  invisível por 30 s
  deu certo   → apaga            fim
  deu erro    → 30 s depois      visível de novo
                                 tentativa 2, 3...
```

CUIDADO

O tempo de invisibilidade precisa ser **maior que a duração da função**; a AWS recomenda seis vezes o tempo limite dela. Curto demais, a mensagem reaparece enquanto ainda está sendo processada, e o pedido roda duas vezes.

CONCEITO AULA 13

A fila de mensagens mortas

Uma mensagem com defeito falharia para sempre, voltando à fila e gastando execuções. A **DLQ**, *dead-letter queue* ou fila de mensagens mortas, resolve: depois de N tentativas, a mensagem é **movida para outra fila**, onde fica guardada para alguém olhar. O pedido não some e não trava os outros.

a regra da loja

```
loja-pedidos        tempo de invisibilidade 30 s
                    máximo de recebimentos: 3
                          |
          3ª falha →      v
loja-pedidos-dlq    guarda por até 14 dias

DLQ com mensagem = alguém precisa olhar
```

PENSE ASSIM

É a caixa de "pedidos com problema" do balcão: o atendente não fica tentando o mesmo pedido a tarde inteira; separa, segue atendendo, e alguém resolve depois.

FASE 2-2 · NA IMAGEM

As três defesas da loja contra a falha

TENTAR DE NOVO

Nova tentativa

Falha passageira, como o pagamento fora do ar por um minuto, se resolve sozinha na segunda ou terceira vez.

SEPARAR

DLQ

Falha permanente, como um pedido com dado inválido, sai do caminho dos outros e fica guardada para análise.

DUPLICAR

Duas zonas

A Lambda e o SQS já rodam em várias zonas de disponibilidade, e a função ficou em duas sub-redes na Aula 12.

Resiliência não é nunca falhar. É falhar de um jeito que não perde dado, não derruba o resto e avisa alguém.

FASE 2-2 · O QUE ACONTECE?

A função consumidora tem tempo limite de 5 s. O que acontece em cada caso?

- a) O pagamento ficou fora do ar por 40 segundos.
- b) Um pedido chegou com quantidade 999 e a função sempre lança erro.
- c) A fila tem tempo de invisibilidade de 3 s.
- d) A DLQ tem 12 mensagens numa manhã de segunda-feira.

a) As mensagens reaparecem depois de 30 s e a segunda ou terceira tentativa passa. Ninguém perde pedido.

b) Três tentativas e a mensagem vai para a DLQ. Os outros pedidos seguem normalmente.

c) Mais curto que a função: a mensagem reaparece no meio do processamento e o pedido roda duas vezes.

d) Doze clientes sem pedido processado. Precisa de alguém olhando, e por isso a fase 2-3 existe.



FASE 2-3 · CONTEÚDO NOVO

Enxergar

Logs, métricas e alarmes: saber que quebrou antes do cliente.

CONCEITO AULA 13

Observabilidade

Observabilidade é conseguir responder **o que está acontecendo** com o sistema olhando para o que ele emite, sem precisar adivinhar. Três sinais: **logs**, o que aconteceu em cada execução; **métricas**, números ao longo do tempo; e **rastros**, o caminho de um pedido entre os serviços. Na AWS, o **CloudWatch** recebe logs e métricas de todos os serviços da loja sem configurar nada.

os três sinais da loja

```
log       "pedido a91f processado"      CloudWatch Logs
          "ERRO pedido 77c0: quantidade"

métrica   mensagens na fila: 340         CloudWatch Metrics
          erros da função: 12 por minuto

rastro    API 3 ms → fila 8 ms → ...     X-Ray (para ir além)
```

PENSE ASSIM

Log é o diário de bordo; métrica é o painel de instrumentos; rastro é o mapa da viagem. O `print` que vocês escrevem na Lambda desde a Aula 7 já é log.

CONCEITO AULA 13

Alarme: a métrica que chama alguém

Um **alarme** do CloudWatch vigia uma métrica e muda de estado quando ela passa de um limite: `OK`, `ALARM` ou `INSUFFICIENT_DATA`. Ao entrar em `ALARM`, avisa por um tópico do **SNS**, *Simple Notification Service*, que entrega por e-mail, SMS ou para outro sistema. Bom alarme vigia o **sintoma que o cliente sente**, não cada detalhe técnico.

os alarmes da loja

sintoma	métrica	limite
pedido perdido de vez	mensagens na DLQ	> 0
pedidos atrasando	mensagens na fila	> 100
a porta recusando pedido	5xx do API Gateway	> 1%

ALARM → tópico loja-alertas → e-mail da equipe

CUIDADO

Alarme que dispara toda hora vira ruído, e ruído é ignorado. Comece com poucos alarmes, ligados a sintomas reais, e um limite que alguém sabe o que fazer ao receber.

FASE 2-3 · FAÇA A CONTA

O que a prática de hoje custa

RECURSO	USO NA AULA	FAIXA GRATUITA
SQS	algumas centenas de mensagens	1 milhão de pedidos por mês
Lambda	algumas centenas de execuções curtas	1 milhão de pedidos e 400 mil GB-s por mês
CloudWatch	logs, métricas padrão e 1 alarme	10 alarmes e métricas básicas
SNS	alguns e-mails	1.000 entregas de e-mail por mês
NAT Gateway ou endpoint de interface	**nenhum**	são pagos por hora: ficam de fora

A CONTA FECHA EM ZERO DE NOVO

O único jeito de gastar hoje seria pôr a porta da fila numa sub-rede privada: aí ela precisaria de um endpoint de interface para chegar ao SQS, e esse cobra por hora. Por isso a função nova fica fora da VPC.

HORA DA PRÁTICA

Agora é com vocês

- Console da AWS na região **N. Virginia (us-east-1)**, a mesma da loja.
- Três fases, cada uma com um checkpoint no CloudShell.
- Duas funções novas: a **porta**, `loja-pedidos`, que põe o pedido na fila, e a **consumidora**, `loja-processa`, que tira e processa.
- A `loja-api` da Aula 12 **não muda**: o catálogo continua síncrono, como deve ser.
- Usem um e-mail que vocês abrem durante a aula: o alarme da fase 3-3 chega nele.

> ORÇAMENTO
> Tudo o que se cria hoje cabe na faixa gratuita. A trava de orçamento da Atividade 1 continua valendo.



FASE 3-1 · PRÁTICA

A fila da loja

A fila com a DLQ, a função porta e a rota POST /pedidos.

FASE 3-1 · NO CLOUDSHELL

Primeiro a DLQ, depois a fila

filas.sh

```
aws sqs create-queue --queue-name loja-pedidos-dlq

DLQ=$(aws sqs get-queue-url --queue-name loja-pedidos-dlq \
  --query QueueUrl --output text)
DLQ_ARN=$(aws sqs get-queue-attributes --queue-url $DLQ \
  --attribute-names QueueArn --query Attributes.QueueArn --output text)

cat > atributos.json <<FIM
{"VisibilityTimeout": "30",
 "RedrivePolicy": "{\"deadLetterTargetArn\":\"$DLQ_ARN\",\"maxReceiveCount\":\"3\"}"}
FIM

aws sqs create-queue --queue-name loja-pedidos --attributes file://atributos.json
```

- A DLQ nasce **antes**, porque a fila principal aponta para ela.
- `maxReceiveCount` 3: na terceira falha, a mensagem muda de fila.
- Tempo de invisibilidade de 30 s: seis vezes o tempo limite de 5 s da consumidora.
- Prefere o console? **SQS › Create queue**, tipo **Standard**, e em **Dead-letter queue** escolha `loja-pedidos-dlq` com **Maximum receives** 3.

FASE 3-1 · NO CONSOLE DA LAMBDA

A porta: lambda_function.py da loja-pedidos

loja-pedidos · lambda_function.py

```python
import json
import os
import uuid

import boto3

sqs = boto3.client("sqs")
FILA = os.environ["FILA_URL"]
CHAVE = os.environ.get("CHAVE_LOJA", "")


def lambda_handler(event, context):
    cabecalhos = event.get("headers") or {}
    if CHAVE == "" or cabecalhos.get("x-api-key") != CHAVE:
        return {"statusCode": 401, "body": '{"erro": "chave ausente ou inválida"}'}

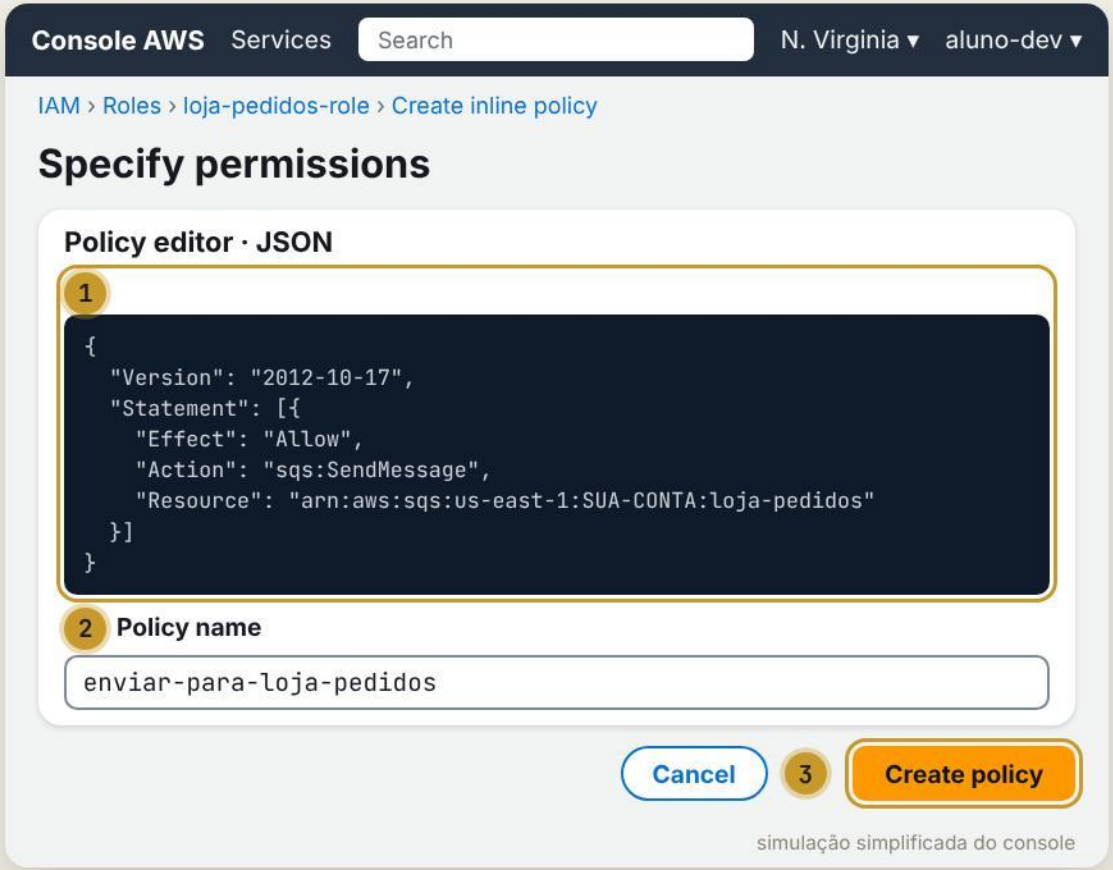
    pedido = json.loads(event.get("body") or "{}")
    pedido["id"] = uuid.uuid4().hex[:8]
    sqs.send_message(QueueUrl=FILA, MessageBody=json.dumps(pedido))

    return {"statusCode": 202,
            "body": json.dumps({"pedido": pedido["id"], "status": "na fila"})}
```

- **Create function**, Python, nome `loja-pedidos`, papel novo com permissões básicas. **Sem VPC**.
- Variáveis de ambiente: `FILA_URL` com a URL da `loja-pedidos`, e a mesma `CHAVE_LOJA` da Aula 9.
- A porta não processa nada: gera o `id`, põe na fila e responde **202** em milissegundos.
- Tempo limite pode ficar nos 3 s padrão: a porta nunca espera ninguém.

FASE 3-1 · NO CONSOLE DO IAM

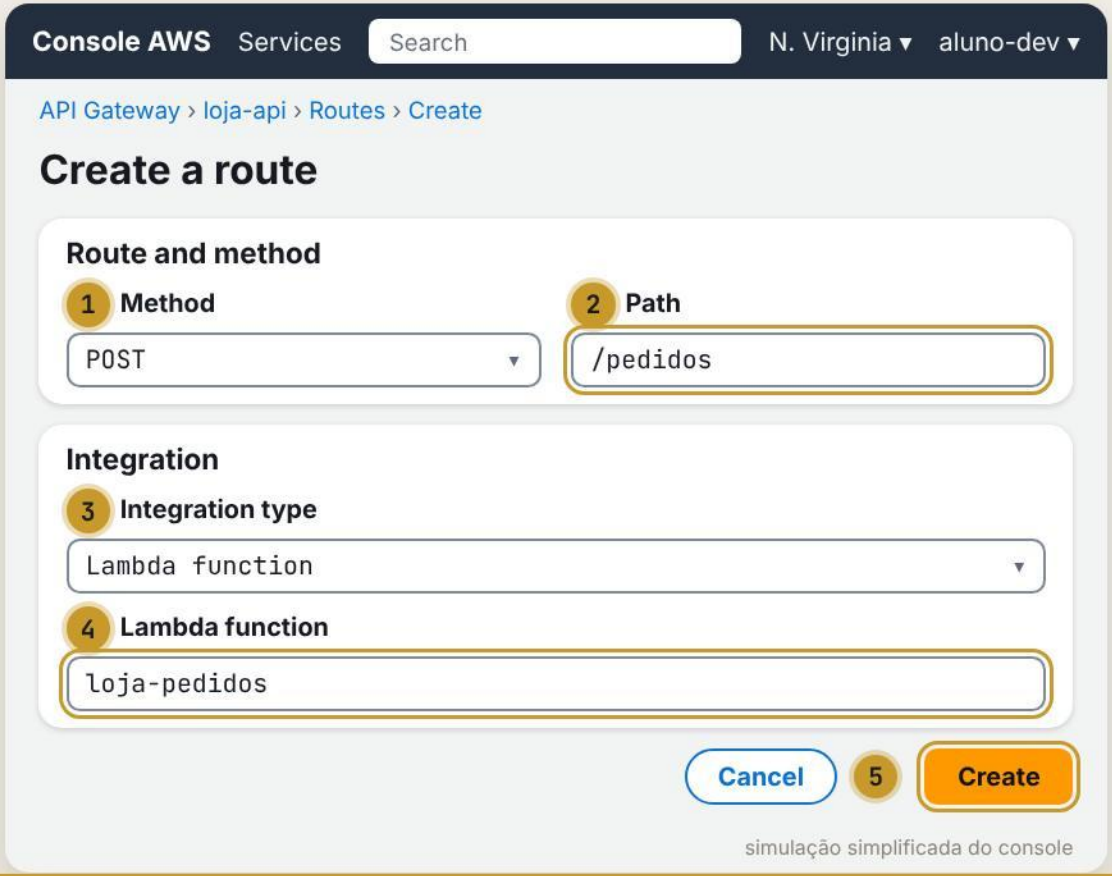
A porta só pode pôr na fila



1. Na função: **Configuration › Permissions** › clique no papel. No IAM: **Add permissions › Create inline policy**, aba **JSON**.
2. Nome `enviar-para-loja-pedidos`.
3. **Create policy**.

Uma ação, uma fila: o menor privilégio da Aula 9. Troque `SUA-CONTA` pelo número de 12 dígitos que aparece no canto do console.

A rota nova



1. Na `loja-api`, **Routes › Create**: método **POST**.
2. Caminho `/pedidos`.
3. Em **Attach integration**: tipo **Lambda function**.
4. Função `loja-pedidos`.
5. **Create**, e depois **Deploy** no estágio **v1**.

FASE 3-1 · CONCLUÍDA

Checkpoint: o pedido entrou na fila

checkpoint

```
$ H="x-api-key: SUA-CHAVE"
$ curl -H "$H" -X POST $API/pedidos -d '{"produto": 2, "quantidade": 1}'
{"pedido": "a91f03c2", "status": "na fila"}
$ FILA=$(aws sqs get-queue-url --queue-name loja-pedidos --query QueueUrl --output text)
$ aws sqs get-queue-attributes --queue-url $FILA --attribute-names ApproximateNumberOfMe
"ApproximateNumberOfMessages": "1"
```

CONFIRA

O `POST` responde **202** com um número de pedido, e a fila mostra **1 mensagem** esperando. Ninguém consome ainda: é a fase 3-2.

TRAVOU NESTA FASE?

`500` com `AccessDenied` no log: falta a política inline no papel, ou o ARN dela está com a conta errada. `404`: faltou o **Deploy** para a v1.



FASE 3-2 · PRÁTICA

O pico na fila

A função consumidora, no máximo duas por vez, e 100 pedidos de uma vez.

A consumidora: lambda_function.py da loja-processa

loja-processa · lambda_function.py

```python
import json
import time


def lambda_handler(event, context):
    for registro in event["Records"]:
        pedido = json.loads(registro["body"])

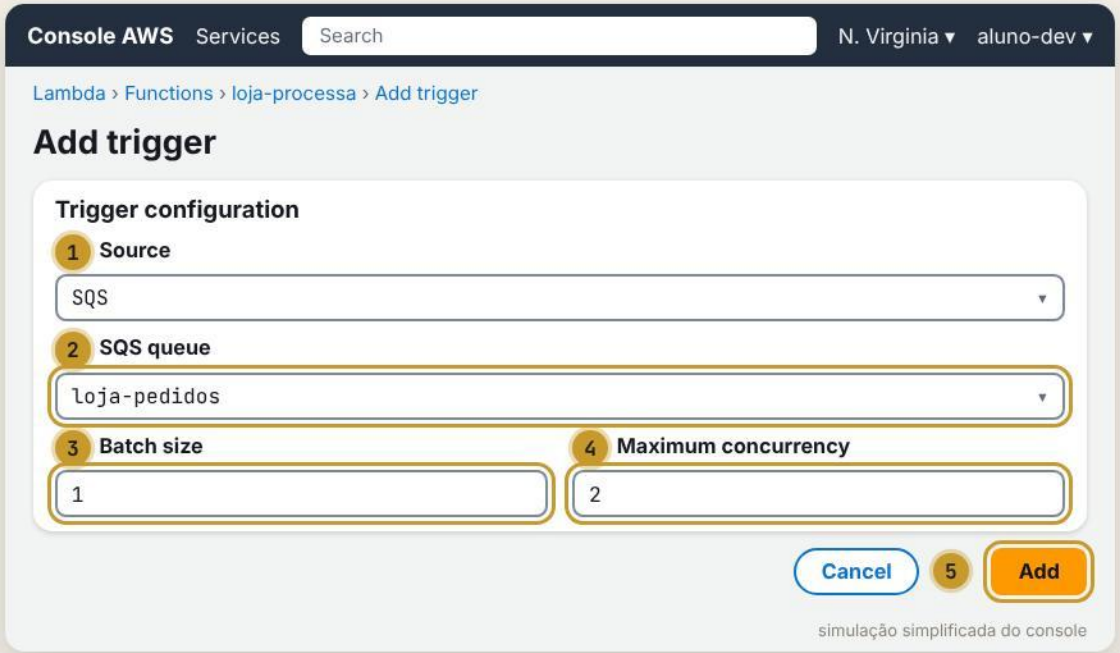
        if pedido.get("quantidade", 1) > 10:
            raise ValueError(f"pedido {pedido['id']}: quantidade acima do estoque")

        time.sleep(1)   # o trabalho lento: pagamento e nota fiscal
        print(f"pedido {pedido['id']} processado")
```

- Função nova `loja-processa`, Python, papel novo. **Tempo limite: 5 s**, em **Configuration › General**.
- No papel dela, a política gerenciada `AWSLambdaSQSQueueExecutionRole`: ler e apagar mensagens da fila.
- O `sleep(1)` faz o papel do pagamento e da nota fiscal: um segundo por pedido.
- Quantidade acima de 10 é o pedido com defeito da fase 3-3. Erro lançado = mensagem volta para a fila.

FASE 3-2 · NO CONSOLE DA LAMBDA

Ligar a fila à consumidora



1. **Add trigger**, origem **SQS**.
2. Fila `loja-pedidos`. A mensagem do checkpoint 3-1 vai ser consumida logo que o gatilho ligar.
3. **Batch size** 1: um pedido por execução, para a falha de um não arrastar os outros.
4. **Maximum concurrency** 2: no máximo duas consumidoras ao mesmo tempo. É o ritmo que a "nota fiscal" aguenta.
5. **Add**.

Quem busca as mensagens é o serviço da Lambda, não a função: ela não precisa de rede até o SQS.

FASE 3-2 · NO CLOUDSHELL

O pico: 100 pedidos de uma vez

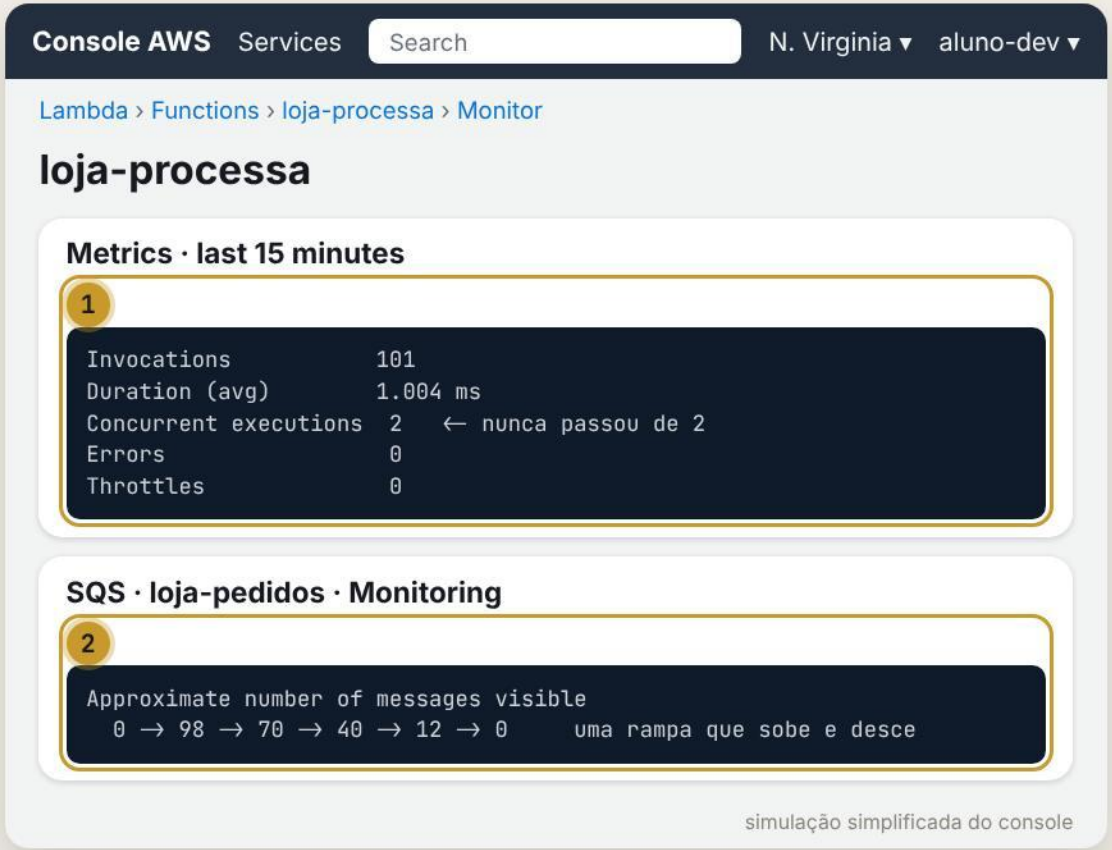
terminal

```
$ P='{"produto": 1, "quantidade": 1}'
$ seq 100 | xargs -P 20 -I{} curl -s -o /dev/null -w "%{http_code}\n" \
    -H "$H" -X POST $API/pedidos -d "$P" | sort | uniq -c
    100 202

$ watch -n 5 aws sqs get-queue-attributes --queue-url $FILA \
    --attribute-names ApproximateNumberOfMessages ApproximateNumberOfMessagesNotVisible
```

- Vinte pedidos em paralelo até completar cem: todos voltam **202** em menos de um segundo cada.
- O `watch` mostra a fila: perto de 100 logo depois, caindo uns 2 por segundo. Em menos de um minuto, zero.
- `NotVisible` fica em 2: são os pedidos sendo processados naquele instante.
- Apareceu **429**? É o limite do estágio da Aula 9 segurando. Em **Protect › Throttling**, suba para rate 50 e burst 100.

O que o pico deixou registrado



1. Na `loja-processa`, aba **Monitor**: concorrência travada em 2, nenhum erro.
2. No SQS, fila `loja-pedidos`, aba **Monitoring**: o pico virou uma rampa.

Valores de exemplo. É o gráfico mais importante da aula: o pico entrou de uma vez e saiu no ritmo combinado, sem nenhum pedido recusado.

FASE 3-2 · CONCLUÍDA

Checkpoint: o pico passou

checkpoint

```
$ aws sqs get-queue-attributes --queue-url $FILA --attribute-names ApproximateNumberOfMe
"ApproximateNumberOfMessages": "0"
$ aws logs tail /aws/lambda/loja-processa --since 10m | grep -c processado
101
```

CONFIRA

A fila voltou a **zero** e o log da consumidora tem **101** linhas de pedido processado: os 100 do pico e o do checkpoint 3-1.

TRAVOU NESTA FASE?

A fila não esvazia: confira se o gatilho está **Enabled** e se o papel da `loja-processa` tem a `AWSLambdaSQSQueueExecutionRole`.



FASE 3-3 · PRÁTICA

O alarme

Um pedido com defeito vai parar na DLQ, e o alarme avisa por e-mail.

FASE 3-3 · NO CLOUDSHELL

O tópico de alertas e o e-mail da equipe

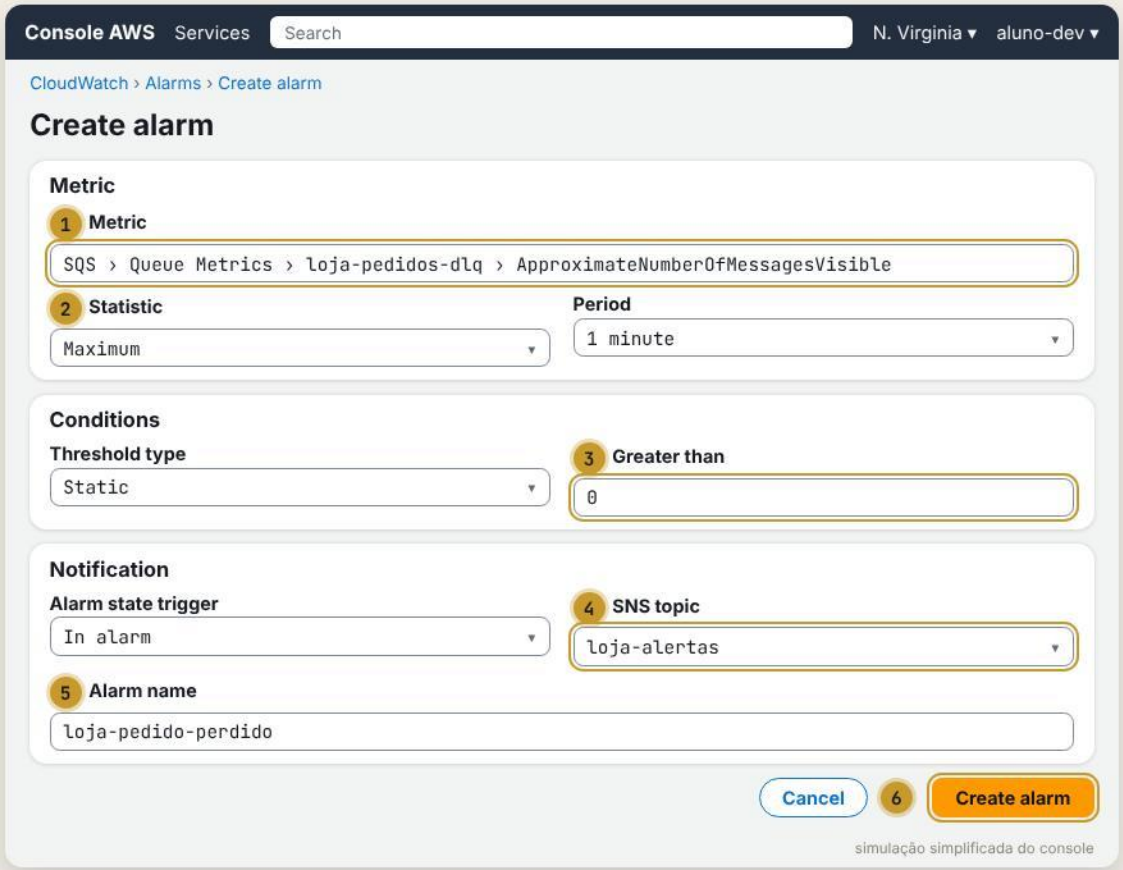
alertas.sh

```
TOPICO=$(aws sns create-topic --name loja-alertas \
  --query TopicArn --output text)

aws sns subscribe --topic-arn $TOPICO \
  --protocol email \
  --notification-endpoint seu-email@exemplo.com
```

- O tópico é o "megafone": quem estiver inscrito recebe.
- Troque o e-mail pelo de vocês. Chega uma mensagem da AWS com **Confirm subscription**: sem clicar, nada é entregue.
- Cada pessoa da equipe pode se inscrever no mesmo tópico.

O alarme do pedido perdido



1. **Select metric › SQS › Queue Metrics**: a métrica `ApproximateNumberOfMessagesVisible` da **DLQ**.
2. Estatística **Maximum**, período de 1 minuto.
3. Condição: maior que **0**. Qualquer pedido na DLQ é um cliente sem pedido.
4. Notificação: estado **In alarm**, tópico `loja-alertas`.
5. Nome `loja-pedido-perdido`.
6. **Create alarm**.

A métrica da DLQ só aparece na lista depois que a fila teve alguma atividade. Se não achar, siga para o próximo slide e volte aqui em dois minutos.

FASE 3-3 · NO CLOUDSHELL

Provocar a falha de propósito

terminal

```
$ curl -H "$H" -X POST $API/pedidos -d '{"produto": 3, "quantidade": 99}'
{"pedido": "77c0e4b1", "status": "na fila"}

$ aws logs tail /aws/lambda/loja-processa --since 5m | grep ERROR
[ERROR] ValueError: pedido 77c0e4b1: quantidade acima do estoque
[ERROR] ValueError: pedido 77c0e4b1: quantidade acima do estoque
[ERROR] ValueError: pedido 77c0e4b1: quantidade acima do estoque
```

- A porta aceita: ela não valida estoque, só registra. O **202** volta normal.
- A consumidora falha **três vezes**, com 30 s entre uma e outra: o tempo de invisibilidade.
- Na terceira, a mensagem vai para a `loja-pedidos-dlq`. Em uns dois minutos o alarme vai para **In alarm** e o e-mail chega.
- Enquanto isso, mandem pedidos normais: eles continuam passando. A falha de um não trava a fila.

FASE 3-3 · CONCLUÍDA

Checkpoint: a loja com fila e alarme

- Filas `loja-pedidos` e `loja-pedidos-dlq`, com 3 recebimentos no máximo.
- Porta `loja-pedidos` respondendo 202 em `POST /pedidos`.
- Consumidora `loja-processa` com no máximo 2 execuções ao mesmo tempo.
- Alarme `loja-pedido-perdido` em **In alarm** e o e-mail na caixa de entrada.

CONFIRA

O pedido 99 está na DLQ, os outros foram processados, e a equipe **ficou sabendo por e-mail**, sem ninguém olhar o console.

TRAVOU NESTA FASE?

O e-mail não chegou: a inscrição está **Confirmed** em **SNS › Subscriptions**? Veja também a caixa de spam. Para o alarme voltar a OK, em **SQS › loja-pedidos-dlq › Purge**.

Um componente desacoplado e um alarme



- Escolher **uma** operação do projeto que pode terminar depois: e-mail, relatório, processamento, integração externa.
- Pôr uma fila com DLQ na frente dela, com porta respondendo 202 e consumidora com concorrência limitada.
- Um alarme ligado a um sintoma real, entregando no e-mail da equipe.
- Um teste de carga pequeno, e o print do gráfico da fila subindo e descendo.

FASE 3-3 · ANTES DE SAIR

Os prints que vão para o relatório

#	PRINT	O QUE PRECISA APARECER
1	SQS, fila principal, aba Dead-letter queue	a DLQ ligada, com máximo de recebimentos
2	Lambda consumidora, gatilho SQS	tamanho do lote e concorrência máxima
3	SQS, aba Monitoring	a rampa da fila subindo e descendo no teste de carga
4	CloudWatch › Alarms	o alarme e o histórico de mudança de estado
5	O e-mail do alarme	o aviso chegando, com data e hora

PODE DEIXAR LIGADO

Fila parada, alarme e tópico não custam nada fora do uso. Tudo isso entra na arquitetura de 19/11. Se o alarme ficar disparando, esvazie a DLQ com Purge.



O MAPA DE HOJE

Mapa completo

Nove fases fechadas: a loja aguentou cem pedidos de uma vez pela fila, um pedido com defeito foi separado sem travar os outros, e a equipe ficou sabendo por e-mail.

ONDE ESTAMOS AGORA

A arquitetura completa

Navegador → S3	a vitrine · 10/09	✔ pronto
API Gateway	rotas, estágio v1, limite e chave · 08/10 e 15/10	✔ pronto
Lambda	as regras da loja · 01/10	✔ pronto
+ classes e ciclo de vida	arquivos baratos com o tempo · 22/10	✔ pronto
Banco NoSQL	os dados saem do código · 29/10	✔ pronto
Rede (VPC)	sub-rede privada e endpoint · 05/11	✔ pronto
Fila, escala e alarme	SQS, DLQ, concorrência e CloudWatch · 12/11	✔ pronto

- Todas as peças da loja existem: vitrine, API, regras, banco, rede, fila e alarme.
- O que é rápido continua **síncrono**; o que é lento passou para a **fila**.
- Em **19/11**, cada equipe apresenta a própria arquitetura funcionando, ao vivo.

FECHAMENTO

O que cada fase construiu

FASE	CONCEITO	O QUE PROVA
1-1	escalar na vertical e na horizontal, elasticidade	o diagnóstico da Aula 12
1-2	concorrência, lei de Little, cota e throttling	a conta das cópias
1-3	acoplamento síncrono e o elo mais fraco	onde quebra
2-1	fila, produtor e consumidor, 202, idempotência	o pico vira fila
2-2	invisibilidade, nova tentativa, DLQ, duas zonas	o que acontece na falha
2-3	logs, métricas, rastros e alarmes	a conta que fecha em zero
3-1	a fila, a porta e a rota	o 202 e a mensagem na fila
3-2	a consumidora e o teste de carga	a rampa no gráfico
3-3	a DLQ e o alarme por e-mail	o aviso chegando sozinho

PRÓXIMA AULA · 19/11

Arquitetura final: o que levar

- **Diagrama** da arquitetura da equipe: cada serviço, a rede, a fila e por onde passa um pedido.
- **Demonstração ao vivo**: um pedido de ponta a ponta e, se a equipe tiver fila, o pico e o alarme.
- **Custo**: a conta do mês num dia normal, e o que mudaria num pico.
- **Relatório** com os prints das Aulas 11, 12 e 13.
- **Arguição**: preparem a resposta para "o que quebra primeiro se o movimento multiplicar por dez?".

ENSAIO

Testem a demonstração no computador de quem vai apresentar, com o CloudShell aberto e a chave da API à mão. Conta de laboratório expira: confiram o acesso na véspera.

PARA ESTUDAR

Referências

BÁSICAS

AMAZON WEB SERVICES. *What is Amazon Simple Queue Service?* Amazon SQS Developer Guide. https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html

AMAZON WEB SERVICES. *Using Lambda with Amazon SQS*. AWS Lambda Developer Guide. https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html

AMAZON WEB SERVICES. *Understanding Lambda function scaling*. AWS Lambda Developer Guide. https://docs.aws.amazon.com/lambda/latest/dg/lambda-concurrency.html

AMAZON WEB SERVICES. *Using Amazon CloudWatch alarms*. Amazon CloudWatch User Guide. https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/AlarmThatSendsEmail.html

COMPLEMENTARES

AMAZON WEB SERVICES. *Using dead-letter queues in Amazon SQS*. Amazon SQS Developer Guide. https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html

AMAZON WEB SERVICES. *Reliability pillar*. AWS Well-Architected Framework. https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html

LITTLE, John D. C. A proof for the queuing formula L = λW. *Operations Research*, v. 9, n. 3, p. 383–387, 1961.

PARA DEIXAR ABERTO

Cola da aula

escalar

```
concorrência = pedidos/s x duração
teto da conta: Service Quotas
passou do teto: throttling, 429

síncrono: o cliente espera a cadeia toda
fila: 202 na hora, trabalho depois
consumidor idempotente: o id manda
```

resistir e enxergar

```
invisibilidade  ⩾ 6 x tempo da função
falhou N vezes  → DLQ

log       o que aconteceu
métrica   o número no tempo
alarme    a métrica que chama alguém
SNS       o megafone: e-mail, SMS
```